



طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

تمرین چهارم

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

فهرست مسائل

۱		مسئله ۱
۱		الف
۱		ب
۱		ج
۲		مسئله ۲
۳		مسئله ۳
۴		مسئله ۴
۵		مسئله ۵

پاسخ مسئله‌ی ۱.

الف

Domain Coupling این مفهوم به میزان وابستگی و ارتباط متقابل بین بخش‌ها یا دامنه‌های مختلف یک سیستم نرم‌افزاری اشاره دارد. به زبان ساده، وقتی دو بخش از سیستم شدیداً به هم متصل باشند (Coupling بالا)، تغییر در یکی باعث خرابی یا نیاز به تغییر در دیگری می‌شود. در معماری‌های نوین (مثل میکروسرویس‌ها)، هدف این است که این وابستگی را به حداقل برسانند تا هر بخش بتواند به صورت مستقل توسعه یافته و تغییر کند.

ب

Data Decomposition تجزیه داده‌ها به معنای شکستن و تقسیم کردن یک پایگاه داده بزرگ و یکپارچه به دیتابیس‌های کوچک‌تر و مجزا است؛ به طوری که هر میکروسرویس یا دامنه فقط به داده‌های مربوط به خودش دسترسی داشته باشد. به زبان ساده، یعنی به جای اینکه همه بخش‌های یک سیستم اطلاعات خود را در یک انبار مشترک بریزند، هر بخش انبار اختصاصی کوچک خودش را داشته باشد تا تداخلی در کار هم ایجاد نکنند.

ج

Event-Driven Communication Pattern یک روش ارتباطی بین سرویس‌های مختلف است که در آن، اجزای سیستم از طریق ارسال و دریافت رویدادها با یکدیگر صحبت می‌کنند. به زبان ساده، وقتی در یک بخش از سیستم اتفاقی می‌افتد (مثلاً خرید کاربر ثبت شد)، آن بخش یک اعلان یا سیگنال به هوا می‌فرستد. سرویس‌های دیگر که به این موضوع علاقه دارند، این سیگنال را دریافت کرده و کار خودشان را انجام می‌دهند، بدون اینکه نیاز باشد مستقیماً با سرویس اولیه تماس بگیرند یا منتظر پاسخ آن بمانند.

پاسخ مسئله‌ی ۲.

مشکل اصلی در این سناریو، تجزیه زودرس سیستم یا همان Premature Decomposition (یا اصطلاحاً Distributed Monolith) است. تیم توسعه پیش از درک کامل بیزینس و مرزهای دامنه مسئله (Bounded Contexts)، سیستم را به میکروسرویس‌های متعدد تقسیم کرده است. این کار بدون وجود نیاز واقعی به مقیاس‌پذیری، تنها پیچیدگی‌های عملیاتی را به شدت افزایش داده است.

شروع مستقیم با معماری میکروسرویس در ابتدای پروژه به دلایل زیر اشتباه است:

- عدم شناخت دامنه: وقتی مرزهای بیزینس شفاف نیست، میکروسرویس‌ها به اشتباه تفکیک می‌شوند. این امر منجر به ایجاد Domain Coupling شدید شده و تغییر در یک ویژگی، نیازمند تغییر همزمان در چندین سرویس و API می‌شود.
- ایجاد منولیت توزیع‌شده: سیستم تمام معایب منولیت (وابستگی شدید) و تمام معایب میکروسرویس (پیچیدگی شبکه، دشواری تست و باگ‌های مرزی) را یکجا دارد.
- کاهش سرعت توسعه: نوشتن تست‌های End-to-End در یک سیستم توزیع‌شده با دیتابیس‌های مجزا بسیار سخت‌تر از یک سیستم یکپارچه است که نتیجه آن افت شدید سرعت تحویل Featureهاست.

رویکرد استاندارد و پیشنهادی، استراتژی Monolith First است:

۱. شروع با یک منولیت ماژولار: سیستم باید در ابتدا به صورت یکپارچه اما با کدهای ماژولار و ساختاریافته توسعه یابد. در این حالت، مرزها در سطح کد (پکیج‌ها یا کامپوننت‌ها) تعریف می‌شوند و ارتباطات به جای شبکه، از طریق فراخوانی توابع داخل حافظه است.
۲. کشف مرزهای دامنه: با رشد پروژه و شناخت دقیق‌تر رفتار کاربران و بیزینس، مرزهای واقعی دامنه‌ها (Subdomains) مشخص می‌شوند.
۳. مهاجرت تدریجی در صورت نیاز: هر زمان که یک ماژول خاص از نظر فنی نیاز به مقیاس‌پذیری مستقل پیدا کرد یا تیم‌های توسعه آن‌قدر بزرگ شدند که روی یک کدبیس تداخل داشتند، آن ماژول به صورت یک میکروسرویس مجزا جدا می‌شود.

پاسخ مسئله‌ی ۳.

تفاوت دو رویکرد بر اساس معیارهای مطرح شده:

۱. Coupling:

- همگام: دارای Coupling بالایی است. سرویس سفارش باید مستقیماً آدرس (IP/URL) و ساختار سرویس‌های پرداخت و ایمیل را بشناسد.
- ناهمگام: دارای Loose Coupling است. سرویس سفارش فقط رویداد را به Message Broker می‌فرستد و هیچ شناختی از گیرنده‌ها ندارد.

۲. Latency:

- همگام: تاخیر کل برابر است با مجموع زمان پاسخ‌دهی تمام سرویس‌ها. سرویس سفارش باید منتظر بماند تا پرداخت و ایمیل کارشان تمام شود که سرعت را پایین می‌آورد.
- ناهمگام: تاخیر برای کاربر به شدت کاهش می‌یابد؛ زیرا به محض ثبت سفارش و ارسال ایونت، پاسخ به کاربر برگشت داده می‌شود و بقیه کارها در پس‌زمینه انجام می‌شود.

۳. Fault Tolerance:

- همگام: پایین است. اگر سرویس ایمیل یا پرداخت در آن لحظه در دسترس نباشد، کل فرآیند ثبت سفارش با خطا مواجه می‌شود.
- ناهمگام: بالا است. اگر سرویس ایمیل خراب باشد، پیام در Queue یا Broker باقی می‌ماند و به محض بالا آمدن سرویس، پردازش می‌شود؛ بدون اینکه تراکنش اصلی خراب شود.

۴. Scalability:

- همگام: به سختی مقیاس می‌شود؛ زیرا فشار کاری مستقیماً به سرویس‌های بعدی منتقل می‌شود و در صورت هجوم کاربران، گلوگاه ایجاد می‌شود.
- ناهمگام: بسیار بالا است. Message Broker مانند یک ضربه‌گیر عمل کرده و پیام‌ها را ذخیره می‌کند؛ سرویس‌ها می‌توانند متناسب با توان خود و به صورت ناهمزمان پیام‌ها را مصرف کنند.

با توجه به تحلیل بالا، بهترین استراتژی ترکیب هر دو رویکرد به شکل زیر است:

۱. ارتباط با سرویس پرداخت با رویکرد همگام:

فرآیند پرداخت یک عملیات حیاتی و وابسته به زمان است. سرویس سفارش باید فوراً متوجه شود که آیا پرداخت با موفقیت انجام شد یا خیر تا وضعیت سفارش را مشخص کند. بنابراین استفاده از یک فراخوانی همگام مانند REST در اینجا منطقی‌تر است.

۲. ارتباط با سرویس ارسال ایمیل با رویکرد ناهمگام:

ارسال ایمیل تأییدیه خرید، تأثیری روی موفقیت آمیز بودن خود فرآیند ثبت سفارش ندارد. کاربر نباید معطل ارسال ایمیل بماند. بنابراین، سرویس سفارش پس از نهایی شدن پرداخت، رویدادی مانند OrderConfirmed را در Message Broker منتشر می‌کند و سرویس ایمیل آن را به صورت ناهمگام برمی‌دارد و پردازش می‌کند.

پاسخ مسئله‌ی ۴.

پاسخ مسئله‌ی ۵.

آ: nginx

کاربرد: به عنوان API Gateway، لود بالانسر و Reverse Proxy عمل می‌کند. درخواست‌های ورودی کاربران را دریافت کرده و به میکروسرویس مربوطه هدایت می‌کند. همچنین وظایفی مثل SSL Termination و کش کردن محتوای استاتیک را انجام می‌دهد.

مثال واقعی: روی یک سرور، فرانت‌اند روی app.example.com و بک‌اند روی api.example.com قرار دارند؛ nginx بر اساس ساب‌دامین، درخواست‌ها را به ترتیب به سرویس فرانت (مثلا React روی پورت ۳۰۰۰) و سرویس API (مثلا روی پورت ۸۰۰۰) هدایت می‌کند.

ب: Prometheus

کاربرد: ابزاری برای مانیتورینگ و جمع‌آوری متریک‌ها به صورت Pull-based است. وضعیت سلامتی، میزان مصرف CPU/RAM و تعداد درخواست‌های هر میکروسرویس را ثبت می‌کند.
مثال واقعی: سرویس پرداخت متریکی به نام successful_payments_total را اکسپوز می‌کند و Prometheus هر ۳۰ ثانیه آن را می‌خواند تا در صورت افت ناگهانی تراکنش‌ها، هشدار صادر کند.

پ: RabbitMQ

کاربرد: یک Message Broker مبتنی بر پروتکل AMQP است که برای ارتباطات ناهمگام و مطمئن بین سرویس‌ها استفاده می‌شود. پیام‌ها را در صف‌ها نگه می‌دارد تا سرویس مقصد آن‌ها را پردازش کند.
مثال واقعی: در یک سیستم فروشگاهی، پس از خرید موفق، سرویس سفارش پیامی به صف shipping-queue می‌فرستد تا سرویس ارسال کالا بعداً آن را بردارد و پردازش کند.

ت: Celery

کاربرد: یک صف وظایف توزیع‌شده برای پایتون است که کارهای سنگین، زمان‌بر یا دوره‌ای را خارج از چرخه اصلی درخواست کاربر اجرا می‌کند.
مثال واقعی: وقتی کاربر درخواست خروجی اکسل از گزارش‌های یک‌ساله خود را می‌دهد، سرویس وب این کار سنگین را به Celery می‌سپارد تا در پس‌زمینه انجام شود و به کاربر پیام گزارش در حال آماده‌سازی است را فوراً نشان می‌دهد.

ث: ELK (Elasticsearch, Logstash, Kibana)

کاربرد: پشته‌ای برای مدیریت، تجمع و تحلیل لوکال و متمرکز لاگ‌ها در سیستم‌های توزیع‌شده است. Logstash لاگ‌ها را جمع‌آوری و فیلتر می‌کند، Elasticsearch آن‌ها را ایندکس و ذخیره می‌کند و Kibana محیط گرافیکی نمایش آن‌هاست.
مثال واقعی: وقتی یک خطای ناشناخته در فرآیند ثبت‌نام رخ می‌دهد، برنامه‌نویس به جای بررسی لاگ‌های تک‌تک سرورها، در Kibana شناسه کاربر را جستجو می‌کند تا تمام لاگ‌های مربوط به او را در میکروسرویس‌های مختلف ببیند.

ج: Redis

کاربرد: یک پایگاه داده درون‌حافظه‌ای بسیار سریع است که بیشتر به عنوان لایه کش، مدیریت نشست‌ها یا Rate Limiter استفاده می‌شود.

مثال واقعی: اطلاعات پروفایل کاربران یا کاتالوگ محصولات پربازدید در Redis کش می‌شود تا نیازی به کوئری زدن مداوم به دیتابیس‌های اصلی و کندتر نباشد.

چ: Kubernetes

کاربرد: پلتفرم ارکستراسیون کانتینرها (Container Orchestration) است. وظیفه مدیریت چرخه حیات، استقرار اتوماتیک، مقیاس‌پذیری خودکار (Auto-scaling) و حفظ پایداری کانتینرهای میکروسرویس‌ها را بر عهده دارد.

مثال واقعی: در زمان کمپین‌های تخفیفی که ترافیک سایت ۱۰ برابر می‌شود، Kubernetes به طور خودکار تعداد Podهای سرویس کاتالوگ را افزایش می‌دهد تا سیستم زیر بار ترافیک ن خوابد.

ح: Kafka

کاربرد: یک پلتفرم توزیع‌شده برای پردازش و انتقال جریان داده‌ها (Event Streaming) با حجم و سرعت بسیار بالا است. بر خلاف واسطه‌های سنتی، پیام‌ها را به صورت پایدار روی دیسک ذخیره می‌کند و چندین مصرف‌کننده می‌توانند تاریخچه داده‌ها را بازخوانی کنند.

مثال واقعی: در سیستم‌های تاکسی اینترنتی، جریان مختصات جغرافیایی (GPS) راننده‌ها به صورت مداوم به Kafka فرستاده می‌شود تا سرویس‌های تخمین قیمت، نقشه و بررسی رفتار راننده همزمان از آن استفاده کنند.

خ: Jaeger

کاربرد: ابزاری برای ردیابی توزیع‌شده (Distributed Tracing) است. مسیر حرکت یک درخواست کاربر را در میان میکروسرویس‌های مختلف دنبال کرده و زمان صرف شده در هر مرحله را به صورت گرافیکی نمایش می‌دهد تا گلوگاه‌ها کشف شوند.

مثال واقعی: اگر لود شدن صفحه سبد خرید ۵ ثانیه طول بکشد، با استفاده از Jaeger مشخص می‌شود که مثلاً ۲.۴ ثانیه از این زمان مربوط به کوئری دیتابیس در سرویس تخفیف بوده است.